Davante

9

BASE DE DATOS

La sentencia SELECT

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. DML: El lenguaje de manipulación de datos 5

/ 3. El comando SELECT 6

/ 4. Consultas simples 7

4.1. Cláusula FROM 7

4.2. Concepto de alias en SQL 7

/ 5. Filtrado de registros: La cláusula WHERE 8

/ 6. Ordenación de registros: La cláusula ORDER BY 9

6.1. Sintaxis 9

6.2. Criterios de ordenación 10

6.3. Orden ascendente y descendente 10

/ 7. Agrupación de registros: La cláusula GROUP BY 11

/ 8. La cláusula HAVING 12

/ 9. Cálculos aritméticos y concatenación de textos 13

© MEDAC ISBN: 978-84-19885-68-5
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 10. Caso práctico 1: “Consultas sobre PEDIDOS y PRODUCTOS” 14

/ 11. Caso práctico 2: “Consultas sobre PEDIDOS y PRODUCTOS” 15

/ 12. Resumen y resolución del caso práctico de la unidad 15

/ 13. Bibliografía 17

OBJETIVOS

Conocer el lenguaje de manipulación de datos (DML).

Aprender a llevar a cabo consultas para extraer información de una base de datos.

Entender el uso de la sentencia SELECT y la construcción de sentencias.

Familiarizarse con las cláusulas pertinentes para filtrar y ordenar registros desde una fuente de datos específica.

Reconocer la manera correcta de construir consultas simples, de agrupación y de resumen en SQL.

/ 1. Introducción y contextualización práctica

El principal objetivo de crear una base de datos es almacenar información para acceder a ella cuando sea necesario. Para esto, se usa **la sentencia SELECT,** que **permite acceder a la información almacenada en la base de datos, seleccionando solo los datos de interés y presentándolos en un orden específico**.

Comprender el uso de esta sentencia y sus vastas posibilidades **es fundamental para la administración y explotación de bases de datos**, sin importar el tipo de base de datos o el SGBD (sistemas de gestión de bases de datos) utilizado. Aunque los SGBD incluyen herramientas gráficas para ayudar en la realización de consultas, es esencial aprender a construir sentencias SQL usando la cláusula SELECT y conocer las opciones que ofrece para filtrar información, ordenar resultados, elegir la fuente de datos, etc. En este tema, empezaremos a estudiar esta cláusula, que será abordada en temas posteriores y se utilizará repetidamente a lo largo del curso.

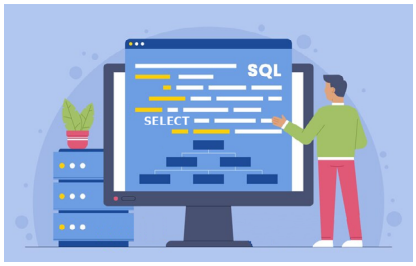

Fig.1. En este tema, se estudiarán algunas operaciones básicas en una BD, como la realización de consultas para extraer información.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Aprendiendo a realizar consultas con SQL"

https://on.soundcloud.com/Vy5KL5KbS9M3cNjB6

/ 2. DML: El lenguaje de manipulación de datos

El lenguaje DML (que es un sublenguaje de SQL) incluye **comandos diseñados para acceder y gestionar la información en las tablas de una base de datos**. Estos comandos permiten **modificar la información existente, agregar nuevos datos o eliminar los no deseados**.

Específicamente, los comandos SQL que forman parte de este lenguaje se muestran en la siguiente imagen:

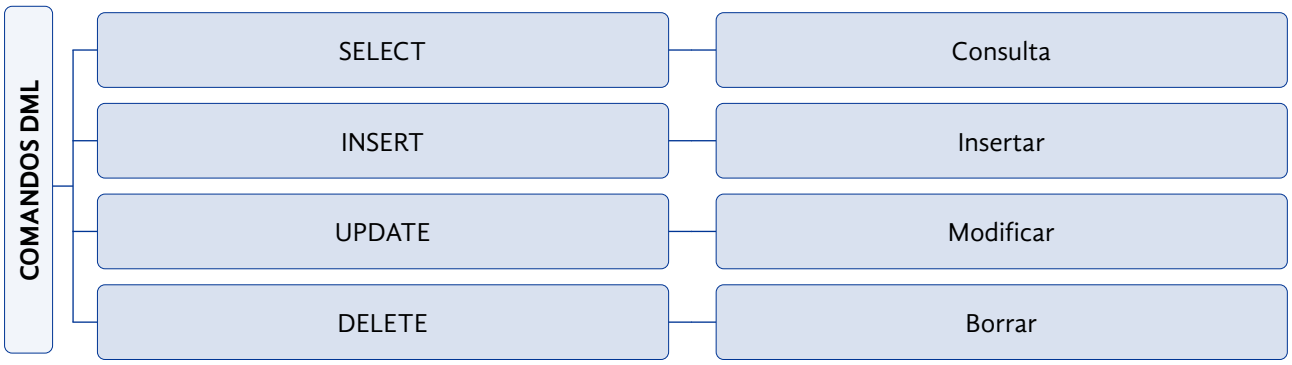

Fig.2. Comandos DML más utilizados .

Aunque se estudiarán en detalle más adelante en el temario, en términos generales, estos comandos se utilizan para las siguientes acciones:

- **SELECT**: Utilizado para consultar información.
- **INSERT**: Empleado para añadir datos a una tabla.
- **UPDATE**: Destinado a modificar datos en una tabla.
- **DELETE**: Comúnmente usado para eliminar filas de una tabla.

Al ejecutar una instrucción DML, el sistema proporcionará al usuario tanto el resultado de la operación como el número de filas que han sido modificadas. Esto permite verificar si la acción se ha realizado con éxito.

```
SQL> select table_name from user_tables;

TABLE_NAME
------------------------------
ASIGNATURAS
DOCENCIA
PLAN_TABLE
PROFESORES
PRUEBA_DEVELOPER
TABLA_CLASES_SALA
TABLA_CLASES_SOCIO

7 filas seleccionadas.
```

Fig.3. Ejemplo de resultado de una consulta DML.

Por otra parte, **una transacción se compone de un conjunto de instrucciones DML que se ejecutan de manera secuencial**. Este enfoque es bastante común, ya que ofrece la posibilidad de revertir la transacción si el resultado no es el esperado.

/ 3. El comando SELECT

El comando SELECT se emplea para realizar consultas en una base de datos, con el objetivo principal de extraer información. Permite llevar a cabo varias acciones, entre las cuales se incluyen:

- **Extraer datos** específicos de ciertas columnas dentro de una misma tabla.
- **Recuperar filas** de una tabla basándose en diversos criterios o filtros de selección.
- **Agrupar los datos** obtenidos según los criterios deseados.
- **Ejecutar operaciones aritméticas y/o concatenaciones** sobre los datos.
- **Extraer información de múltiples tablas**, un aspecto que se analizará en temas posteriores.

Para construir una sentencia SELECT, debemos utilizar una sintaxis similar a la siguiente:

```
SELECT [ALL | DISTINCT] <campos>
FROM <tablas>
[WHERE <condición> [AND | OR <condición>]]
[GROUP BY <nombre_campo>]
[HAVING <criterios_de_agrupación>]
[ORDER BY <nombre_campo>|<índice_campo> [ASC|DESC]];
```

En esta sintaxis, podemos destacar los siguientes elementos:

- **Comando SELECT**: Después de este comando, se especifican los datos que se desean consultar, indicando los nombres de las columnas de las que queremos obtener información.

  Es posible **seleccionar todas las columnas de una tabla utilizando el asterisco** (*). Además, se pueden usar las palabras clave ALL y DISTINCT de manera opcional:

  » **ALL**: Permite seleccionar todas las filas, incluidas las duplicadas.

  » **DISTINCT**: Descarta los valores repetidos evitando duplicados.

- **Cláusula FROM**: Esta cláusula especifica las tablas de las que se van a extraer los datos.

Las cláusulas opcionales, indicadas entre corchetes (WHERE, GROUP BY, etc.), se explicarán con mayor detalle a lo largo de este tema.

```
SELECT * FROM CLIENTES WHERE NOMBRE = 'JAVIER';
```

Fig.4. Ejemplo de uso de la cláusula SELECT.

/ 4. Consultas simples

Para realizar cualquier consulta en SQL, es fundamental emplear el comando SELECT acompañado de la cláusula FROM.

4.1. Cláusula FROM

La cláusula FROM señala las tablas de las que se extraerán los datos, es decir, aquellas que contienen las columnas especificadas en la sentencia SELECT. Es una cláusula que siempre acompañará a SELECT, es decir, **toda consulta SELECT debe tener una cláusula FROM**.

Si se incluyen varias tablas en la consulta, estas deben separarse por comas. Es lo que se llama una **consulta combinada**. Además, opcionalmente, se puede especificar el propietario de cada tabla usando la sintaxis 'USUARIO. TABLA'. Esto es importante, ya que pueden existir tablas con el mismo nombre pertenecientes a distintos usuarios, y es necesario diferenciarlas en la consulta.

```
SELECT IDEMPLEADO,
       NOMBRE,
       APELLIDO,
       DEPARTAMENTO
FROM EMPLEADOS;
```

Fig.5. Ejemplo de consulta simple.

4.2. Concepto de alias en SQL

En ocasiones, los nombres de las columnas o de las tablas que queremos consultar pueden ser extensos, complejos o poco descriptivos. Al escribir una consulta, especialmente si esta utiliza muchas columnas o tablas, esto puede dificultar notablemente la lectura y comprensión. **Para facilitar la legibilidad de una consulta, SQL nos permite asignar nombres o alias a las columnas o tablas que se utilizan en ella**.

En la siguiente imagen, podemos ver un ejemplo de cómo asignar el alias 'f_mat' a la columna 'FECHA_DE_ MATRICULACION' de la tabla Coches.

```
SELECT FECHA_DE_MATRICULACION "f_mat" FROM COCHES;
```

Fig.6. Creación de alias 'f_mat' para la columna 'FECHA_DE_MATRICULACION'.

En el caso de las tablas no es necesario indicar el alias entre comillas.

```
SELECT * FROM CLIENTES cli
```

Fig.7. Creación de alias 'cli' para la tabla CLIENTES.

/ 5. Filtrado de registros: La cláusula WHERE

La sentencia SELECT por sí sola permite acceder a los datos de todas las columnas de una o varias tablas. No obstante, a menudo solo nos interesan los datos presentes en ciertas filas específicas.

Para restringir o filtrar la obtención de datos a las filas que nos interesan, empleamos la cláusula **WHERE**, la cual **nos permite establecer una condición específica que debe cumplirse**. La sintaxis para usar esta cláusula es la siguiente:

```
SELECT [ALL | DISTINCT] <campos>
FROM <tablas>
[WHERE <condición> [AND | OR <condición>]]
```

Para establecer las condiciones a cumplir, pueden usarse diversos operadores o expresiones como son: **BETWEEN**, **=**, **>**, **<**, etc. Observemos este vídeo para conocer los más habituales.

Vídeo 1. "Operadores y expresiones"
https://bit.ly/3CSUO9g

```
SELECT MATRICULA, FABRICANTE, COLOR
FROM COCHES
WHERE FABRICANTE = 'BMW';
```

Fig.8. Ejemplo de uso de WHERE, en el que se especifica que se extraigan datos (matrícula, fabricante, color) de coches cuyo fabricante sea BMW.

```
SELECT * FROM CLIENTES WHERE NOMBRE = 'JAVIER';
```

Fig.9. En esta consulta, se seleccionan todos los campos de la tabla CLIENTES en los que la columna NOMBRE sea 'JAVIER'.

En la siguiente imagen, se presenta un ejemplo de consulta diseñada para seleccionar información sobre ciertos productos del almacén que se han registrado en un intervalo de fechas específico. Para ello, se han utilizado las cláusulas BETWEEN y AND.

```
SELECT IDPRODUCTO,
    NOMBRE,
    DESCRIPCION,
    FECHA
FROM ALMACEN,
WHERE FECHA BETWEEN '15/05/2019' AND '15/05/2020';
```

Fig.10. Ejemplo de uso de WHERE con las cláusulas BETWEEN y AND.

Enlaces de interés...

En este vídeo podrás encontrar ejemplos de varias consultas en las que se hace uso de estas cláusulas: https://www.youtube.com/watch?v=np6PH_vs-GI&list=PLU8oAlHdN5Bmx-LChV4K3MbHrpZKefNwn&index=3

/ 6. Ordenación de registros: La cláusula ORDER BY

A menudo resulta útil que la información obtenida mediante una consulta SELECT **se muestre al usuario organizada según un criterio específico**. Para lograr esto, se emplea la cláusula **ORDER BY**. De esta manera, el resultado será más fácil de interpretar para cualquier usuario.

6.1. Sintaxis

Una sentencia que incluya ORDER BY tiene la siguiente sintaxis:

```
SELECT [ALL | DISTINCT] <campos>
FROM <tablas>
[WHERE <condicion> [AND | OR <condición>]]
[ORDER BY columna1[ASC | DESC],..., columnaN[ASC | DESC] ]
```

```
SELECT NOMBRE,FECHA_ALTA
FROM CLIENTES
WHERE FECHA_ALTA < '2020-01-01'
ORDER BY FECHA_ALTA;
```

Fig.11. Ejemplo de sentencia con ORDER BY.

Audio 1. "Criterio para ejecutar sentencia con ORDER BY"
https://bit.ly/2P2y5iW

Es importante tener en cuenta que no es posible ordenar y agrupar (concepto que veremos más adelante con la cláusula Group By) la información utilizando cualquier tipo de datos. **La ordenación y agrupación solo se admiten mediante tipos de datos específicos**.

Los **tipos de datos admitidos para la ordenación y agrupación son** los siguientes:

- **Carácter (VARCHAR o CHAR)**: Utilizados para almacenar cadenas de texto, estos tipos de datos son muy comunes en bases de datos y permiten ordenar la información alfabéticamente.
- **Número**: Este tipo de datos incluye enteros y decimales, y se utiliza para almacenar valores numéricos. La ordenación con números permite organizar la información en orden ascendente o descendente según su valor.
- **Fecha**: Este tipo de datos se emplea para almacenar fechas y horas. La ordenación por fecha es útil para organizar registros cronológicamente, ya sea en orden ascendente (de la fecha más antigua a la más reciente) o descendente (de la fecha más reciente a la más antigua).

Por tanto, al realizar consultas SQL que incorporen ORDER BY, es esencial prestar atención a los tipos de datos de las columnas involucradas.

¡! Sabías que...

Existen funciones adicionales que permiten transformar tipos de datos para simplificar las operaciones de ordenación y agrupación. No obstante, estas prácticas deben emplearse con cuidado, ya que pueden impactar negativamente en el rendimiento de la consulta.

6.2. Criterios de ordenación

Ahora que conocemos la cláusula que debemos utilizar para ordenar los resultados obtenidos, debemos conocer cómo debemos indicar el criterio que se quiere utilizar para ordenar los datos. Para ello, **tras escribir la cláusula, se debe escribir el criterio de ordenación**.

Este criterio de ordenación puede implicar a una única columna o a varias columnas de la tabla, o incluso a un campo calculado durante la ejecución.

En la siguiente imagen, podemos ver un ejemplo de sentencia SELECT en la que se realiza la ordenación de los datos con la cláusula ORDER BY sobre la columna 'product_id'.

```
SELECT product_id,product_name
FROM products
ORDER BY product_id;
```

Fig.12. Ejemplo de uso de las cláusulas SELECT y ORDER BY.

Además de las opciones que hemos mencionado para establecer el criterio de ordenación, la cláusula ORDER BY nos ofrece también la **posibilidad de indicar el orden a través del índice de la columna**, es decir, la posición numérica que tiene el campo por el que queremos que se ordene dentro de la lista de campos de la selección.

Para mayor claridad de lo explicado, en la siguiente imagen, podemos ver un ejemplo de este caso.

```
SELECT product_id,product_name
FROM products
WHERE category_id = 50
ORDER BY 1;
```

Fig.13. Ejemplo de uso de las cláusulas SELECT, WHERE y ORDER BY.

Evidentemente, el valor del índice indicado debe ser menor o igual que el número de parámetros incluidos en la cláusula SELECT, ya que, si no, se produciría un error al ejecutar la consulta.

Además, dado que se puede ordenar utilizando varias columnas, la cláusula ORDER BY permite combinar el uso de los nombres de las columnas con el uso de índices.

Enlaces de interés...

En este vídeo, se muestran algunos ejemplos de consultas en las que se hace uso de la cláusula ORDER BY para realizar la ordenación: https://www.youtube.com/watch?v=yZk9NdxFUrk&list=PLU8oAlHdN5Bmx-LChV4K3MbHrpZKefNwn&index=4

6.3. Orden ascendente y descendente

Para ordenar la información seleccionada al ejecutar nuestra consulta, es necesario indicar qué columnas queremos utilizar para la ordenación. Pero, además de esto, **es conveniente indicar si la información debe ordenarse de forma ascendente o descendente**.

Las expresiones **ASC** o **DESC** nos permiten indicar que **el orden se realizará de forma ascendente o descendente**, respectivamente. Por defecto, **el sistema aplicará una ordenación ascendente**, por lo que la cláusula ASC apenas se utiliza.

La sintaxis sería utilizar la cláusula ORDER BY acompañada de ASC o DESC después del nombre o índice de la columna utilizada para la ordenación. En las siguientes imágenes, se muestra un ejemplo.

```
SELECT product_id,product_name
FROM products
WHERE category_id = 50
ORDER BY 1 DESC;
```

Fig.14. Ejemplo de uso de la cláusula DESC.

```
SELECT product_id,product_name
FROM products
WHERE category_id = 50
ORDER BY product_id ASC;
```

Fig.15. Ejemplo de uso de la cláusula ASC.

Es importante tener en cuenta el comportamiento que la cláusula ORDER BY tiene ante los valores **NULL**. Los valores NULL no pueden cumplir con las condiciones de una consulta como el resto de datos, ya sea ante la cláusula WHERE o la cláusula ORDER BY.

Existen dos cláusulas que nos permiten indicar la posición de los valores nulos. **NULLS FIRST** indica que **los valores nulos deben figurar al principio**. La cláusula **NULLS LAST** permite indicar que **los valores nulos deben ubicarse al final**. Por defecto, **los valores nulos serán ubicados al final**, por lo que esta cláusula apenas se utiliza. En la siguiente imagen podemos ver un ejemplo de su uso.

```
SELECT product_id,product_name
FROM products
WHERE category_id = 50
ORDER BY product_id NULLS FIRST;
```

Fig.16. Ejemplo de uso de NULLS FIRST.

/ 7. Agrupación de registros: La cláusula GROUP BY

En numerosas ocasiones, resulta útil agrupar los datos según un campo específico. Para ello, se utiliza las cláusulas **GROUP BY** y **HAVING**, las cuales estudiaremos a continuación. Mediante su uso se pueden obtener **subtotales de un campo determinado**, es decir, un resumen de los datos, como veremos en este apartado.

La cláusula **GROUP BY** se emplea frecuentemente junto con las cláusulas que hemos visto hasta ahora, proporcionando la capacidad de **agrupar los datos según una columna específica**.

```
SELECT employee_number,salary,dept_id
FROM employees
GROUP BY dept_id;
```

Fig.17. Ejemplo de uso de la cláusula GROUP BY.

Además, es importante mencionar las **funciones de agregado**, que complementan la funcionalidad obtenida con la agrupación de registros.

Las consultas que incluyen funciones de agregado **permiten realizar cálculos por grupos**, es decir, los registros se agrupan según un criterio determinado y, una vez agrupados, se pueden ejecutar acciones como calcular medias, contar totales, valores máximos y mínimos de un campo específico, etc. Las **funciones de agregado** se muestran en la siguiente imagen:

Función	Descripción
AVG	Valor promedio de un campo
COUNT	Número total de registros de un campo
SUM	Suma de valores de un campo
MAX	Valor máximo de un campo
MIN	Valor mínimo de un campo

Tabla 1. Funciones de agregado.

En la siguiente imagen, se muestra un ejemplo de aplicación de la función de agregado SUM.

```
SELECT fabricante, SUM(precio) AS suma
FROM coches
GROUP BY fabricante
ORDER BY suma;
```

Fig.18. Ejemplo de uso de la función de agregado SUM para obtener la suma de precios de coches por fabricante.

/ 8. La cláusula HAVING

La cláusula **HAVING se emplea en consultas de agrupación para establecer criterios o condiciones que deben cumplirse para obtener la información**. Puede considerarse como un sustituto de la cláusula WHERE en este tipo de consultas, aunque, en ciertas situaciones, ambas cláusulas pueden usarse en una misma sentencia.

Es importante destacar que **la cláusula HAVING se aplica únicamente a las columnas que están presentes en la cláusula GROUP BY**.

En el ejemplo de la página anterior, en el caso de querer obtener el coste medio de ciertos fabricantes de coches, se utilizaría la cláusula de la imagen siguiente.

```
SELECT OrderID,SUM(Quantity)
FROM OrderDetails
GROUP BY OrderID
HAVING OrderID = 10248 OR OrderID = 10249;
```

Fig.19. Ejemplo de uso de HAVING.

En la siguiente imagen, vemos otro ejemplo de sentencia, que produciría el mismo resultado que el de la imagen anterior.

```
SELECT OrderID,SUM(Quantity)
FROM OrderDetails
WHERE OrderID = 10248 OR OrderID = 10249
GROUP BY OrderID;
```

Fig.20. Sentencia similar a la de la imagen anterior.

Los SGBD suelen tener herramientas gráficas que facilitan la realización de estas consultas complejas. Sin embargo, es esencial conocer la sintaxis para poder aplicarla cuando sea necesario y para entender las instrucciones generadas por el sistema.

Enlaces de interés...

Este vídeo te permitirá profundizar tus conocimientos sobre el uso de la cláusula GROUP BY para realizar la agrupación de registros: https://www.youtube.com/watch?v=TPn1200-fbc&list=PLU8oAlHdN5Bmx-LChV4K3MbHrpZKefNwn&index=5

Las cláusulas WHERE y HAVING pueden usarse juntas en una misma sentencia. Por ejemplo, WHERE puede excluir ciertas filas de los grupos antes de que HAVING aplique una condición a los grupos restantes. De esta manera, WHERE realiza un primer filtrado de filas en la tabla, sobre las cuales se aplicará posteriormente la cláusula HAVING.

/ 9. Cálculos aritméticos y concatenación de textos

Dentro de una consulta, **los operadores matemáticos (+, -, * y /) nos permiten realizar cálculos sobre los datos originales de la tabla sin que estos sean modificados**.

Enlaces de interés...

Existen varias páginas web que permiten probar sentencias sobre bases de datos de prueba. Un ejemplo de ello es: https://www.w3schools.com/sql/trysql.asp?filename=trysql_asc

Para manejar estos cálculos en la consulta, podemos asignarles un nombre o alias, facilitando así su manipulación. En la siguiente imagen, podemos ver un ejemplo de uso de alias utilizando la **cláusula AS**.

```
SELECT nombre, precio, precio*1.16 AS "Precio + IVA"
FROM ARTICULOS;
```

Fig.21. Ejemplo de cálculo dentro de una sentencia SELECT.

```
SELECT ProductID, Price*1.21 AS "IVA incluido"
FROM Products
ORDER BY ProductID DESC;
```

Fig.22. Ejemplo de cálculo dentro de una sentencia SELECT.

Como es habitual, los operadores de multiplicación y división tienen prioridad ante la suma y la resta. En caso de operaciones de igual preferencia, se aplica por orden de izquierda a derecha. El uso de paréntesis permite regular el orden de aplicación.

Además, **la cláusula SELECT permite concatenar** (es decir, unir) **textos**. El **operador de concatenación** utilizado es (||). En la siguiente imagen, se puede ver un ejemplo de su uso para unir las columnas 'tipo' y 'modelo'. Además, se le asigna un alias.

```
SELECT tipo, modelo, tipo || '-' || modelo "Clave Pieza"
FROM PIEZAS;
```

Fig.23. Ejemplo de uso del operador de concatenación.

```
SELECT CustomerID, CustomerName ||' '|| Address, City
FROM Customers
ORDER BY CustomerID;
```

Fig.24. Ejemplo del operador de concatenación donde se unen dos columnas mediante un espacio en blanco.

Tipo	Modelo	Clave Pieza
AR	1	AR-1
AR	2	AR-2
AR	**3**	**AR-3**
BI	**4**	**BI-1**

Tabla 2. Posible resultado obtenido al ejecutar la consulta con las columnas 'tipo' y 'modelo' del ejemplo anterior.

/ 10. Caso práctico 1: "Consultas sobre PEDIDOS y PRODUCTOS"

Planteamiento: Seguimos ayudando a nuestro amigo a implementar la base de datos para su tienda de informática. En el proceso, se han implementado ya varias tablas, y nos pide ayuda para desarrollar las consultas necesarias para obtener información de dos de esas tablas.

Nº_pedido	Fecha_pedido	Cliente	Fab	Producto	Cant	Importe

Tabla 3. Columnas de la tabla PEDIDOS.

Id_fab	Id_producto	Descripción	Precio	Existencias

Tabla 4. Columnas de la tabla PRODUCTOS.

Nudo: ¿Qué sentencias SQL le sugerirías para poder extraer la siguiente información de la base de datos?

1. Listado con todos los productos, mostrando para cada uno el id_fab, el id_producto, la descripción y el precio.
2. Listado de todos los pedidos realizados por el cliente 2, extrayendo toda la información almacenada de los mismos.

Desenlace: En base a lo estudiado en este tema, las siguientes sentencias permitirían extraer los datos que necesita nuestro amigo:

1. SELECT id_fab, id_producto, descripcion, precio FROM PRODUCTOS;
2. SELECT * FROM PEDIDOS WHERE cliente = '2';

/ 11. Caso práctico 2: "Consultas sobre PEDIDOS y PRODUCTOS"

Planteamiento: Nuestro amigo está agradecido por la ayuda que le hemos prestado anteriormente, pero necesita pedirnos ayuda nuevamente para realizar consultas más específicas sobre las tablas PEDIDOS y PRODUCTOS.

Nº_pedido	Fecha_pedido	Cliente	Fab	Producto	Cant	Importe

Tabla 5. Columnas de la tabla PEDIDOS.

Id_fab	Id_producto	Descripción	Precio	Existencias

Tabla 6. Columnas de la tabla PRODUCTOS.

Nudo: ¿Con qué sentencias SQL obtendríamos la siguiente información que necesita?

1. Consultar los datos de los pedidos de más de 100 €.
2. Calcular cuántos pedidos hay de cuantía superior a 1.000 €.
3. Obtener el precio medio de los productos del fabricante con id_fab = "medac".
4. Consultar la fecha del pedido más antiguo.

Desenlace: En base a lo estudiado en este tema, las siguientes sentencias permitirían extraer los datos que necesita nuestro amigo:

1. SELECT (*) FROM PEDIDOS WHERE importe > 100;
2. SELECT COUNT(*) AS cuenta_pedidos_superiores FROM PEDIDOS WHERE importe > 1000;
3. SELECT AVG(precio) AS precio_medio FROM PRODUCTOS WHERE id_fab = 'medac';
4. SELECT MIN(fecha_pedido) AS primer_pedido FROM PEDIDOS;

/ 12. Resumen y resolución del caso práctico de la unidad

Hemos empezado el tema explicando los diferentes comandos que forman parte del sublenguaje de manipulación de datos DML.

A continuación, hemos abordado el comando SELECT, que nos permite realizar consultas de datos, incluyendo las cláusulas que permiten definir el origen de los datos, filtrarlos, ordenarlos y agruparlos.

A continuación podrás ver el esquema resumen del tema:

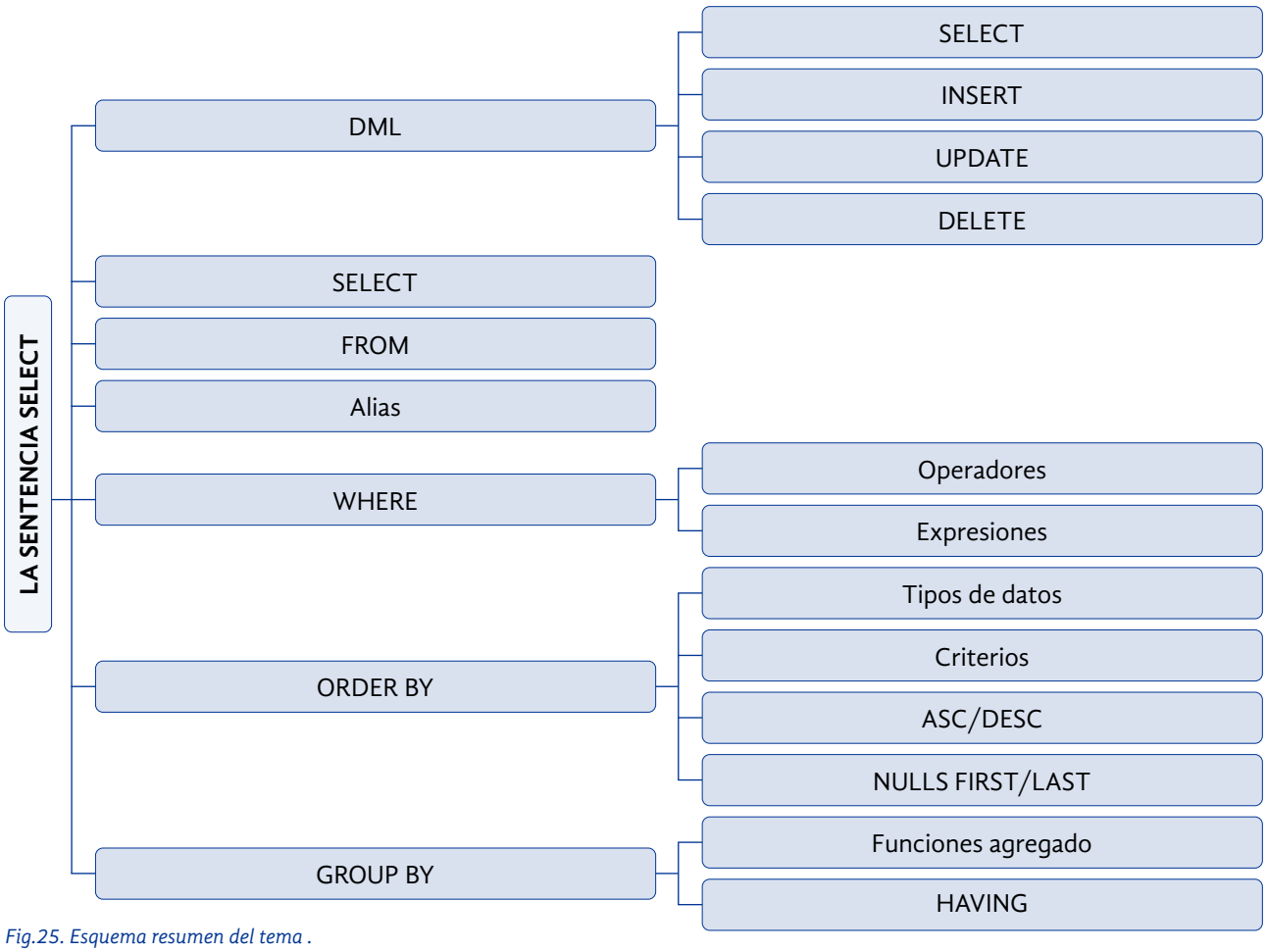

Fig.25. Esquema resumen del tema .

Resolución del caso práctico de la unidad

Cada uno de los perfiles que hemos identificado en la tienda informática de nuestro amigo necesitarán realizar consultas de datos, filtrado, agrupación de registros, etc. Para cada perfil se han detectado los siguientes tipos de consulta:

- **Recursos humanos**: Consultas para obtener información sobre los empleados, sus salarios, fechas de incorporación, condiciones laborales, etc.
- **Dependiente de tienda**: Consultas sobre stock de almacén, precio de los artículos, historial de ventas de un cliente determinado, etc.
- **Gestor de almacén**: Información acerca de los productos con poco stock, así como movimientos de entrada y salida de productos del almacén.
- **Comercial de ventas**: Datos de los clientes como historial de compras, precios, condiciones, ofertas, etc.

En cuanto a las cláusulas que estos perfiles podrían utilizar, serían perfectamente válidas todas las cláusulas. Pero como ejemplo podríamos indicar que Recursos Humanos podría querer calcular el salario medio de los departamentos, utilizando para ello la cláusula GROUP BY, que también podría venirle bien al comercial para saber qué clientes hacen pedidos de mayor importe.

En definitiva, todos los perfiles podrían beneficiarse de todas y cada una de las cláusulas combinadas con el comando SELECT.

/ 13. Bibliografía

Connolly, T., y Begg, C. (2005). *Sistemas de Bases de Datos (5ª ed.)*. Madrid, España. Addison – Wesley.

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos*. Addison Wesley.

López, I.; Castellano, M. J. y Ospino, J. (2011). *Bases de datos.* Garceta.

Oppel, A. (2009). *Databases A Beginner's Guide*. McGraw-Hill Education.

Pérez Marqués, M. (2016). *Administración básica de bases de datos con Oracle 12c SQL*. Madrid, España: Alfaomega.

Sánchez, G. C. (2001). *Sistemas gestores de bases de datos.* Paraninfo.